Buch der Begierde

Softwareanalyse & Qualitaetsverbesserung

Multi-Agenten-Analyse

Explorer-Agent | Review-Agent | Coder-Agent

Projekt	Buch der Begierde v0.6.0
Repository	github.com/astonbrown/buch-der-begierde
Architektur	Vanilla HTML/JS/CSS, ES6 Module, kein Build-Step
Dateien	29 Quelldateien (7 HTML, 19 JS, 9 CSS)
Gesamtzeilen	7.285 LOC
Analysedatum	14.10.2026

1. Zusammenfassung

Die Multi-Agenten-Analyse hat insgesamt **26 Probleme** identifiziert: **4 kritisch** (XSS-Schwachstellen und fehlende Supabase-Einbindung), **8 hoch** (Sicherheit, Architektur, Performance), **9 mittel** (Code-Qualitaet, Wartbarkeit) und **5 niedrig**. **10 konkrete Fixes** wurden durch den Coder-Agent implementiert und deployt.

Kategorie	Kritisch	Hoch	Mittel	Niedrig	Gesamt
Sicherheit	2	2	2	0	6
Code-Qualitaet	0	3	5	2	10
Performance	0	2	3	1	6
Wartbarkeit	0	3	3	1	7
Bugs & Logik	2	3	3	2	10
Gesamt	4	8	9	5	**26**

2. Kritische Probleme (sofort beheben)

S1: Supabase Credentials hardcoded

Datei: js/config.js

Supabase URL und ANON_KEY sind vollstaendig im Quellcode sichtbar. Der JWT-Token kann extrahiert werden, um direkte DB-Zugriffe zu versuchen.

Fix: Umgebungsvariable via .env + Server-Proxy; RLS-Policies validieren; Key rotieren.

S2-S4: XSS via innerHTML mit Nutzereingabe

Datei: toast.js, app-lesen.js, app-archiv.js

Drei Stellen verwenden innerHTML mit ungefilterten Nutzerdaten (node.name, message). Persistierter XSS ueber Memory-Knoten moeglich.

Fix: Alle drei Stellen auf textContent/createElement umgestellt. DOMPurify empfohlen.

B1: Supabase CDN fehlt auf Hauptseiten

Datei: lesen.html, hoeren.html, schreiben.html

Die drei wichtigsten Seiten laden das Supabase-SDK nicht. DB.init() schlaegt fehl, faellt auf LOCAL_-Fallback zurueck. Persistenz ist kaputt.

Fix: Supabase CDN script-Tag auf allen drei Seiten hinzugefuegt.

B3: 312 vs 7 Seiten Diskrepanz

Datei: config.js + story-data.js

CONFIG.STORY_TOTAL_PAGES behauptet 312, aber story-data.js definiert nur 7. Anzeige springt von "312" auf "7" - verwirrend fuer Nutzer.

Fix: STORY_TOTAL_PAGES auf 7 korrigiert, HTML-Placeholder angepasst.

---

3. Hoch-priorisierte Probleme

S5: Fehlende SRI-Integritaets-Hashes

Datei: archiv.html, analyse.html

Font Awesome und Supabase ohne integrity-Attribut von CDN geladen.

Fix: SHA-384-Hash und crossorigin="anonymous" hinzufuegen.

S6: Cookie ohne Secure-Flag

Datei: js/db.js

Cookie wird ohne Secure-Flag gesetzt, ueber HTTP mitlesbar.

Fix: Bedingtes Secure-Flag bei HTTPS hinzugefuegt.

Q1: Monolithische index.html (1199 Zeilen)

Datei: index.html

~600 Zeilen inline CSS + ~400 Zeilen inline JS. Nicht cachebar, doppelter Code.

Fix: Inline CSS/JS in separate Dateien auslagern.

Q3: Duale Design Systeme

Datei: index.html vs css/variables.css

CSS-Variablen zweimal mit unterschiedlichen Werten. Landing Page und Lesen-Seite haben verschiedene Paletten.

Fix: Einheitliches Design System: Landing Page soll variables.css importieren.

B2: Versions-Inkonsistenz

Datei: config.js vs README vs HTML

config.js=0.5.0, README=0.5.2, HTML=0.6.0. Drei verschiedene Versionsnummern.

Fix: Auf 0.6.0 vereinheitlicht; CONFIG.VERSION als Single Source of Truth.

B4: Tote ID-Referenzen im Archiv

Datei: js/app-archiv.js

Referenzen auf IDs die nicht in archiv.html existieren. Archiv-Seite zeigt keine Daten.

Fix: IDs korrigiert: archivNodeList, archivNodeCount etc.

P1-P2: Canvas-Render ungebremst

Datei: js/echo.js

60fps-Render bei inaktivem Tab. getBoundingClientRect pro Frame, Math.random 2880x/Sekunde.

Fix: document.hidden-Check hinzugefuegt; BoundingRect cachen empfohlen.

M1-M3: Dateien ueber 300-Zeilen-Grenze

Datei: index.html, components.css, lesen.css

Drei Dateien uebertreten die 300-Zeilen-Empfehlung deutlich (533, 388, 1199).

Fix: Nach Feature aufteilen (buttons.css, fragments.css, toast.css).

4. Implementierte Fixes (10/10)

Der Coder-Agent hat alle 10 Prioritaets-Fixes implementiert und deployt.

#	Fix	Datei	Beschreibung
1	S2	js/toast.js	XSS: innerHTML -> createElement + textContent
2	S3	js/app-lesen.js	XSS: innerHTML -> createElement + textContent
3	S4	js/app-archiv.js	XSS: innerHTML -> createElement + textContent
4	B1	lesen.html + 2	Supabase CDN script-Tag hinzugefuegt
5	B2	js/config.js	VERSION: 0.5.0 -> 0.6.0
6	B3	js/config.js	STORY_TOTAL_PAGES: 312 -> 7
7	B4	js/app-archiv.js	Tote ID-Referenzen korrigiert
8	S6	js/db.js	Cookie: Secure-Flag bei HTTPS
9	P1	js/echo.js	Canvas: document.hidden-Check
10	P5	js/navigation.js	Scroll: rAF-Throttle hinzugefuegt

5. Architektur-Uebersicht

Das Projekt folgt einem sauberen Schichtenmodell mit klarer Trennung von Eintrittspunkten, Feature-Modulen, Kern-Modulen und Utilities. Alle Module nutzen das Singleton-IIFE-Pattern mit Object.freeze fuer Immutabilitaet und ein konsistentes Event-Emitter-System.

Schicht	Module	Abhaengigkeiten
Utility	config, toast, atmosphere, navigation	config.js (root)
Core	db, memory, story, echo	Utility-Layer
Feature	fragments, io, qr, hoeren, schreiben	Core + Utility
Entry Points	app-landing, app-lesen, ...	Feature + Core + Utility
Data	story-data.js	Keine (reine Daten)

6. Roadmap

PHASE 1: Sofort umsetzen (1-2 Tage)

- XSS-Schwachstellen beheben (IMPLEMENTIERT)
- Supabase CDN auf allen Hauptseiten laden (IMPLEMENTIERT)
- Versionsnummer vereinheitlichen (IMPLEMENTIERT: 0.6.0)
- Story-Seitenanzahl korrigieren (IMPLEMENTIERT: 7 Seiten)
- Archiv-IDs korrigieren (IMPLEMENTIERT)
- Cookie Secure-Flag bei HTTPS (IMPLEMENTIERT)
- Canvas-Pause bei inaktivem Tab (IMPLEMENTIERT)
- Scroll-Event drosseln (IMPLEMENTIERT)

PHASE 2: Mittelfristig (1-2 Wochen)

- SRI-Integritaets-Hashes fuer CDN-Ressourcen hinzufuegen
- Design System vereinheitlichen: index.html auf variables.css migrieren
- Inline CSS aus index.html in css/landing.css auslagern (~600 Zeilen)
- Event-System als gemeinsames EventEmitter-Mixin extrahieren (5x dupliziert)
- Memory-Knoten-Renderer zentralisieren (app-lesen + app-archiv)
- Eingabe-Logik (io.js + schreiben.js) refaktorieren
- Canvas-Performance: getBoundingClientRect cachen, Gradient reusen
- components.css aufteilen: buttons.css, fragments.css, toast.css, cards.css
- maxlength auf textarea setzen (z.B. 500 Zeichen)
- Alle oeffentlichen API-Funktionen mit try/catch absichern

PHASE 3: Langfristige Optimierung (1-3 Monate)

- Build-System einfuehren (Vite/esbuild) fuer Tree-Shaking, Minifizierung, Source Maps
- Supabase-Anon-Key in Umgebungsvariable auslagern + Server-Proxy
- MemoryEngine: maximale Knotenanzahl limitieren, Batch-Decay optimieren
- Story Engine: weitere Seiten implementieren (aktuell 7 von 312)
- QR-Scanner: echte QR-Code-Bibliothek statt Simulation
- Unit-Test-Suite aufbauen (Vitest oder Jest)
- E2E-Tests mit Playwright fuer alle 6 Seiten
- Docker-Container fuer Ollama-Integration
- Lighthouse-CI fuer Performance-Regression
- i18n-System fuer mehrsprachige Erweiterung (derzeit nur Deutsch)